AUFGABENSTELLUNG

Allgemeine Informationen

Die Aufgabenstellung ist als *Einzelarbeit* innerhalb von 2,5 Stunden zu lösen. Wie im realen Programmieralltag ist es erlaubt, das Internet zur Recherche zu verwenden. Bei Fragen zur Problemstellung, wende dich bitte an einen der KNAPP-Betreuer.

Einleitung

KNAPP zählt zu den führenden Technologieunternehmen für Automatisierungslösungen und Software für Distribution und Produktion. Kunden auf der ganzen Welt aus unterschiedlichen Branchen wie zum Beispiel Gesundheitswesen, Onlinehandel, Lebensmittelhandel, Mode- und Textil-Branche bis hin zum Automotive-Bereich vertrauen auf die intelligenten Lösungen von KNAPP. Jede dieser Branchen hat dabei ihre Besonderheiten, denen KNAPP durch maßgeschneiderte Systemlösungen gerecht wird.

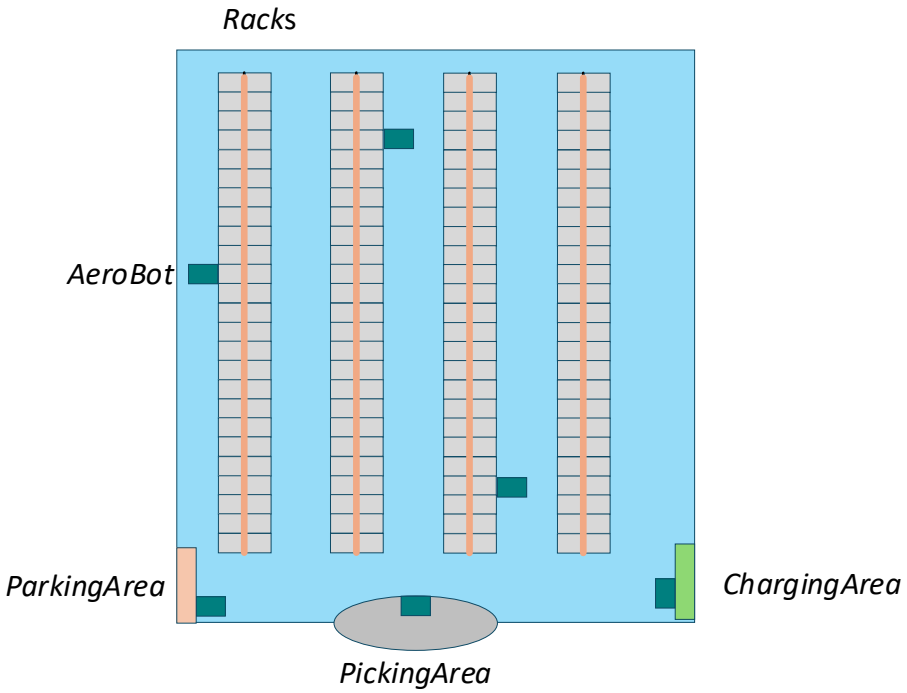
Aufgabe

KNAPP installiert seit kurzem Logistiksysteme die aus AeroBots, selbstständig fahrenden und kletternden Robotern, bestehen. Zusammen mit dem Regalsystem und dem Kommissionierbereich, als AeroBot System, bietet er unseren Kunden effiziente Logistiklösungen, die individuell anpassbar sind, was Prozesse, Regal und Größe betrifft. Unsere Kunden nutzen diese Systeme zur Abarbeitung zur effizienten Abwicklung von Kundenbestellungen in ihren Lagern.

Deine Aufgabe ist es, den Algorithmus für die Steuerung und Koordination der AeroBots zu implementieren.

Die Aufgabe ist eine Optimierungslösung. Versuche zuerst die Aufgabe zu lösen und dein Ergebnis danach Schritt für Schritt zu verbessern.

Das AeroBot System besteht aus:



- Den *AeroBots*, den Robotern selbst mit Akku, die fahren, klettern und Behälter Laden und Lagern können.
- Den *Racks* (Regalen), mit Lagerorten (*RackStorageLocation*) in mehreren Ebenen (*level*) auf denen jeweils ein Behälter (*container*) stehen kann.
- Den *Containern* (Behälter), in denen jeweils ein Produkt in unbegrenzter Menge (Vereinfachung für den Contest) aufbewahrt ist.
- Der *PickingArea* (Kommissionierbereich), wo eine Kundenbestellung mit dem vom AeroBot bereitgestellten Produkt abgewickelt wird. Der Kommissionierbereich hat mehrere Arbeitsplätze, an denen die Bearbeitung stattfindet.
- Der *ChargingArea*, in der Du den Akku eines oder mehrerer AeroBots gleichzeitig aufladen kannst.
- Der *ParkingArea*, in der alle AeroBots anfangs starten und in der Du ein oder mehrere AeroBots parken kannst damit sie keine Energie verbrauchen.

Ein Kunde bestellt ein Produkt (Vereinfachung) in einer Kundenbestellung. Du musst alle Kundenbestellungen abarbeiten und jeweils einen Behälter mit dem benötigten Produkt mit den AeroBots aus dem Regal holen, im Kommissionierbereich bearbeiten, und danach wieder in ein Regal lagern.

Anfangszustand

- Alle zu bearbeitenden Kundenbestellungen erhältst du mit der Methode *warehouse.getAllOrders()*.
- Alle Behälter mit den Produkten sind im Regal eingeräumt und verfügbar. Du erhältst alle Behälter mit der Methode *warehouse.getAllContainers()* oder einen passenden Behälter mit *warehouse.findAvailableContainers(productCode).*
- Alle AeroBots sind im System in der ParkingArea verfügbar und voll aufgeladen. Du erhältst die AeroBots mit der Methode *warehouse.getAllAeroBots()*.

Die Abarbeitung einer Kundenbestellung erfolgt in folgenden Schritten:

- Nimm die nächste zu bearbeitende Kundenbestellung
- Wähle einen passenden Behälter
- Wähle einen passenden AeroBot
- Plane die Fahrt des AeroBot zum Regal ein
- Plane das Klettern des AeroBot auf die Ebene des Behälters ein
- Plane das Laden des Behälters auf den AeroBot ein
    - Damit ist der Behälter für diesen AeroBot exklusiv reserviert
    - Damit ist der Lagerort für diesen Plan exklusiv reserviert
- Plane das Klettern des AeroBots ein
    - AeroBots können nur am Boden (Ebene 0) fahren
- Plane die Fahrt des AeroBot zum Kommissionierbereich ein
- Plane das Kommissionieren ein
- Wähle einen freien Lagerort für den Behälter aus
- Plane die Fahrt des AeroBot zum Regal ein
- Plane das Klettern auf die Ebene des freien Lagerorts ein
- Plane das Lagern des Behälters ein
    - Damit ist der Lagerort für diesen Plan exklusiv reserviert

- Führe die eingeplanten Schritte aus bis der (erste) AeroBot alles abgearbeitet hat
    - *warehouse.executeTicksUntilFirstBotToFinish*()
    - Damit stehen der Behälter und die reservierten Lagerorte von abgearbeiteten Plänen wieder für andere AeroBots zur Verfügung.

Die Aufgabe ist Runden-basiert. Du planst Aktionen für ein oder mehrere AeroBots ein und lässt dann den KNAPP – Code die Aktionen ausführen (warehouse.*executeTicksUntilFirstBotToFinish oder warehouse.executeOneTick*). Die Methode des KNAPP Code kehrt zurück, sowie ein AeroBot alle für ihn eingeplanten Aktionen ausgeführt hat.

Solltest du zur Optimierung Aktionen für mehrere AeroBots eingeplant haben, werden diese zu diesem Zeitpunkt (tick) angehalten und beim nächsten *execute...* weiter ausgeführt.

Du kannst die benötigte Zeit und Energiemenge für eine Aktion erfragen (*warehouse.calculate…*).

ALLGEMEINES

- Alle Kundenbestellungen stehen von Anfang an zur Verfügung und müssen der Reihe nach abgearbeitet werden.
    - Für optimierte Lösungen, siehe auch Anmerkungen im Abschnitt Optimierungen.
- Alle Kundenbestellungen können vollständig bearbeitet werden.
- Es befindet sich immer nur ein Produkt in unbegrenzter Stückzahl in einem Behälter
- Einige Produkte sind in mehreren Behältern vorhanden.
- Die Fahrwege selbst werden durch den KNAPP Code errechnet, es gibt keine Kollisionen der AeroBots am Boden.
- Mehrere AeroBots können sich am selben Wegpunkt (*Waypoint*) befinden, der KNAPP Code führt die eingeplanten Aktionen der AeroBots am Wegpunkt (Klettern) in Reihenfolge des Eintreffens am Wegpunkt aus.
- Ein Behälter wird mit dem Einplanen des Ladens für einen AeroBot reserviert und ist erst mit dem ausgeführten Lagern wieder für andere AeroBots verfügbar.

WICHTIGE METHODEN

`warehouse.getAllOrders()`

Liefert die Liste der Kundenbestellungen (Order).

`warehouse.getAllAeroBots()`

Liefert eine Collection aller AeroBots im System.

`order.getProductCode()`

Liefert den *ProductCode* des Produktes, das für eine Kundenbestellung benötigt wird.

`warehouse.findAvailableContainers( string productCode )`

Liefert eine Collection aller Behälter, in denen dieses Produkt aufbewahrt ist.

`aeroBot.planMoveToWayPoint( WayPoint wayPoint )`

Plant die Fahrt des AeroBot von der aktuellen Position zur angegebenen Position ein.
Solltest du die Dauer der Fahrt im Vorhinein wissen wollen, erhälts du diese mit `warehouse.calculateMoveToWayPoint( WayPoint from, WayPoint to ).`

`aeroBot.planClimb( int level )`

Plant das Klettern auf den angegebenen Level ein.

`aeroBot.planLoadContainer( Container container )`

Plant das Laden des angegeben Behälters von der aktuellen Planposition des AeroBots ein
Der Container und der Lagerort sind ab jetzt exklusiv für diesen AeroBot reserviert.

`aeroBot.planPick( Order order )`

Plant die Kommissionierung der Kundenbestellung mit dem am AeroBot befindlichen Behälter.

`warehouse.findEmptyRackStorageLocations()`

Gibt eine Sammlung der freien und nicht reservierten Lagerorte zurück.

`aeroBot.planStoreContainer()`

Plant das Lagern des angegebenen Behälters an der aktuellen Planposition des AeroBots ein.
Der Lagerort ist ab jetzt exklusiv für diesen AeroBot reserviert.

aeroBot.planStartCharge()

Plant das Aufladen des AeroBot ein. Der AeroBot lädt sich voll auf und steht dann wieder zur Verfügung.
Das Aufladen ist nur an der Ladestation möglich.

pickingArea.getCurrentOrders()

Für Optimierungen.
Gibt die Liste der derzeit an der PickingArea zur Kommissionierung aktiven Kundenbestellungen zurück. Diese Kundenbestellungen kannst du kommissionieren.
Andere AeroBots, die Du in die PickingArea steuerst, warten bis die Kundenbestellungen, für die die AeroBots Behälter geholt haben, zum Kommissionieren aktiv gesetzt wurden.

warehouse.executeOneTick()
warehouse.executeTicksUntilFirstBotToFinish()

Lässt entweder eine Runde (Zeitscheibe) im der Simulation vergehen und arbeitet die eingeplanten Aktionen ab, oder lässt so viele Runden vergehen, bis für einen AeroBot alle eingeplanten Aktionen ausgeführt sind.
AeroBots, die noch eingeplante Aktionen haben, pausieren diese und arbeiten diese dann während des nächsten *executeTick…* weiter ab.

warehouse.calculate…()

Mit den *calculate*… Methoden kannst Du für eine optimierte Lösung die Kosten von einzelnen Aktionen vorab durch den KNAPP Code berechnen lassen.

Kundenbestellung (Order)

Eine Kundenbestellung die ausgeliefert werden soll. Sie beinhaltet genau ein Produkt (*productCode*).

Produkt (Product)

Ein Produkt ist das Erzeugnis, das der Kunde bestellt. Die Produkte werden von Erzeugern an das Lager geliefert, dort eventuell zwischengelagert, für einen Auftrag kommissioniert und an den Kunden versendet.

Ein Produkt ist als dessen Produktcode, einem String, dargestellt.

Regale (Racks)

Ein Regal mit Lagerorten in mehreren Ebenen (*level*) auf denen jeweils ein Behälter (*container*) stehen kann

Ein Regal besteht aus

- Dem Wegpunkt an dem es steht (Das Rack selbst ist ein Wegpunkt, den der AeroBot anfahren kann).
- Den Lagerorten in mehreren Ebenen auf denen die Behälter gelagert werden können.

Behälter (Container)

Ein Behälter dient zur Aufbewahrung eines Produktes in unbegrenzter Stückzahl.

Ein Behälter besteht aus

- Einem Code zur Identifikation (`container.getCode()`).
- Dem Lagerort an dem er steht (`container.getCurrentLocation()`)
- Dem Produkt, das er beinhaltet (`container.getProductCode()`)

LagerOrt (Location)

Ein Lagerort gibt einen Platz für Behälter innerhalb des Systems an.

Ein Lagerort besteht aus

- Dem Typen des Lagerortes, Regal oder AeroBot (`location.getType()`)
- Dem Wegpunkt an dem er sich befindet (`location.getWaypoint()`)

- Der Ebene auf der er sich befindet (`location.getLevel()`)

PickingArea

Der Bereich, in dem die Kundebestellungen mit den Produkten in den, von den AeroBots bereitgestellten, Behältern kommissioniert werden.

Details für eine optimierte Lösung

Der KNAPP Code weist der *PickingArea* Kundenbestellungen der Reihe nach zu.

Es sind mehrere Arbeitsplätze innerhalb der *PickingArea* verfügbar. Alle Kundenbestellungen, die der *PickingArea* zugewiesen sind, können kommissioniert werden. Damit ist die Reihenfolge für die Bearbeitung der Kundenbestellungen aufgeweicht.

Aktuell der PickingArea zugewiesene Kundenbestellungen bekommst Du mit *pickingArea.getCurrentOrders*() .

ChargingArea

Der Bereich in dem Du die Batterie der AeroBots aufladen kannst.

ParkingArea

Der Bereich in dem Du AeroBots parken kannst. Hier verbrauchen sie auch keine Energie.

AeroBot

Roboter mit Akku, der fahren, klettern und Behälter (Container) Laden und Lagern kann.

MUSS-BEDINGUNGEN

- AeroBots können nur am Boden (level 0) zu einem anderen Wegpunkt fahren.
- Ein Lagerort kann zu einem Zeitpunkt nur von einem Plan genutzt werden.
- Der AeroBot muss für die eingeplante Aktion genügend Energie haben.
- Gleichzeitiges Klettern am selben Regal ist nicht möglich, wird aber durch den KNAPP Code automatisch hintenangestellt. Die AeroBots die nicht klettern können, pausieren, bis sie an der Reihe sind.

OPTIMIERUNGEN

Versuche zuerst die Aufgabe so einfach wie möglich zu lösen. Danach kannst du deine Aufgabe weiter optimieren

- Nutze mehrere AeroBots.
- Verwende den „besseren“ Behälter, wenn es mehrere für dasselbe Produkt gibt.
- Suche Kombinationen aus AeroBot, Behälter und Rück-Lagerort mit möglichst kurzen Fahr- und Kletterwegen.
- Der KNAPP Code weist der PickingArea Kundenbestellungen der Reihe nach zu.
    - Es sind mehrere Arbeitsplätze verfügbar. Alle Kundenbestellungen, die der PickingArea zugewiesen sind, kannst Du kommissionieren.
- Aktuell der PickingArea zugewiesene Kundenbestellungen bekommst Du mit *pickingArea.getCurrentOrders()*

BEWERTUNGSSCHEMA

- Es werden nur auf den Bewertungsserver hochgeladene Ergebnisse gewertet.
- Die Resultate werden am Server, mit den dort hinterlegten Algorithmen, errechnet.
- Die Punkte errechnen sich aus
    - Der benötigten Gesamtzeit zur Abarbeitung der Kundenbestellungen (in ticks).
    - Einem Penalty pro nicht abgearbeiteter Kundenbestellung.
    - dem Abgabezeitpunkt Deines besten Ergebnisses (früher ist besser)

Die Gewichtung zwischen Abgabezeitpunkt und restlichen Kosten ist dabei so gewählt, dass in der Regel eine bessere Lösung auch bei späterer Abgabe ein besseres Ergebnis bedeutet.

Bei mehreren Abgaben (Uploads) zählt immer die beste Abgabe zu dem Zeitpunkt, an dem diese getätigt wurde. Wenn Du nach dem Upload einer Lösung weiterarbeitest und durch Deine Änderungen das Ergebnis schlechter wird, bleiben spätere Abgaben unberücksichtigt.

ABGABEMODUS

Zur Beurteilung des Ergebnisses muss die vom KNAPP-Code automatisch erzeugte Datei `upload-xxxxx.zip` über die Abgabeseite hochgeladen werden. In diese Datei wird dein Source automatisch mitverpackt. Dieser Source muss das Ergebnis erzeugt haben. KNAPP behält sich hier Kontrollen vor.

Du kannst alle Code Teile modifizieren, der Abgabeserver interpretiert die Ergebnisdatei (`result.csv`) und diese darf daher im Format nicht verändert werden.

UPLOAD WEBSITE

Unmittelbar nach dem Upload über *https://coding-contest.knapp.at* erhältst du ein detailliertes Feedback zu deiner Abgabe.

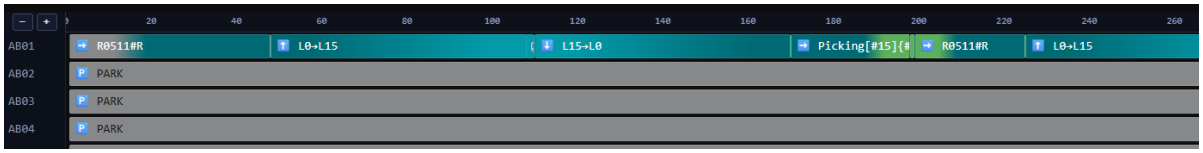
Im Feedback siehst du die Probleme, die noch in deiner Abgabe vorhanden sind, und deinen Score für diesen Upload. Je *niedriger* dieser Wert ist, desto besser ist das Ergebnis.

TIPPS

- Versuche mit deiner Software das Ergebnis zuerst mit einfachen Strategien zu verbessern und arbeite danach an weiteren Optimierungen.
- Schau dir den von uns vorgegeben Code an und prüfe, ob du Teile wiederverwenden oder erweitern kannst.
- Du kannst alle Teile des Source-Codes verändern. Die Abgabedatei muss dem vorgegebenen Format entsprechen.
- Lade öfters hoch - es wird nur die beste Abgabe bewertet.

VISUALISIERUNG

Der KNAPP Code erzeugt eine grafische Übersicht der AeroBots und ihrer Aktionen:



Diese Übersicht findest Du in der Datei *aerobot-visualization.html* im selben Verzeichnis wie Deine Upload-Dateien.
Sie zeigt Dir in den Zeilen die einzelnen AeroBots, und nach rechts in der Zeit alle Aktionen, die diese ausführen.

CODE-DETAILS

- Name und Institution setzen.
- Einsprungspunkt: `Solution::run()`
- Die Klassen im Package `core` sind KNAPP Hilfsklassen und sind für die Lösung nicht von Bedeutung.